Entwurf von Algorithmen: Backtracking bzw. das Backtrack-Prinzip

Dozent: Prof. Dr. Michael Eichberg
Kontakt: michael.eichberg@dhbw.de , Raum 149B

Version: 2.1.0.1

Folien: https://delors.github.io/theo-algo-backtracking/folien.de.md.html
https://delors.github.io/theo-algo-backtracking/folien.de.md.html.pdf
Kontrollfragen: https://delors.github.io/theo-algo-backtracking/kontrollfragen.de.md.html
Fehler melden: https://github.com/Delors/delors.github.io/issues
KI Verwendung: Bei der Erstellung der Unterlagen wurden KI Assistenten (insbesondere Claude aber ggf. auch ChatGPT, Ollama mit Gemma/LLama/Qwen oder OpenCode mit Kimi/Qwen/Deepseek...) unterstützend eingesetzt. Dies erfolgte insbesondere zur Unterstützung bei der Generierung von Grafiken (d. h. SVG Dateien), oder um sich Übersichtstabellen generieren zu lassen. Weiterhin wurde KI zur allgemeinen Qualitätssicherung eingesetzt. Inhalte, die ggf. von der KI vorgeschlagen wurden, wurden im Falle der Übernahme explizit validiert und angepasst.

Backtracking

Neben der dynamischen Programmierung ist das Backtrack-Prinzip ein weiteres grundlegendes Verfahren zur Lösung von Problemen.

- Backtracking ist ein Verfahren, das in vielen Algorithmen zur Anwendung kommt. Insbesondere, wenn kein effizienterer Algorithmus bekannt ist, als *alle möglichen Lösungen auszuprobieren*.
- Backtracking ist eine systematische Methode, um alle möglichen Lösungen eines Problems zu finden. Es ist eine Art von rekursivem Durchsuchen, bei dem Teillösungen zu Gesamtlösungen erweitert werden.
- Backtracking erlaubt ggf. Heuristiken, um die Suche zu beschleunigen.

  Weder die Komplexitätsklasse noch die Korrektheit ändert sich dadurch.
- Viele NP-harte Probleme werden mit Backtracking gelöst.[1]

Backtracking führt eine erschöpfende Suche durch, um eine Lösung zu finden. Kann aber auch direkt genutzt werden, um ggf. alle Lösungen zu finden.

Backtracking ist in Prolog inherent vorhanden, da Prolog auf dem Prinzip des Backtrackings basiert, weswegen Prolog für die Lösung solcher Probleme gut geeignet ist.

[1] NP-harte und NP-vollständige Probleme sind solche Probleme, die schwierig sind zu lösen. Hier reicht es zu wissen, dass es Probleme gibt, die nicht in polynomieller („vernünftiger“) Zeit gelöst werden können.

Beispiel: Das 4-Damen Problem (konzeptuell)

Ziel ist es vier Damen auf einem Schachbrett so zu platzieren, dass keine Dame eine andere Dame schlagen kann.

Eine Lösung für ein 4x4 Schachbrett

	1	2	3	4
1			D	
2	D			
3				D
4		D		

Hinweis

Es gibt eine geschlossene Lösung für das Problem. Backtracking wird hier nur als Beispiel für das Verfahren verwendet.

```
1  procedure findeStellung(i : integer)  // i: Spalte
2      j := 0                            // j: Zeile
3      repeat
4          { wähle nächste Zeile j }
5          if  Dame an Position i / j bedroht
6              keine bisher platzierte Dame then
7              { platziere Dame in Feld i / j }
8              if i = 4 then
9                  { Lösung gefunden }
10                 { Ausgabe der Lösung }
11             else
12                 findeStellung(i + 1) // rek. Aufruf
13             { entferne Dame aus Spalte i und Zeile j } // zurücksetzen des Zustands
14     until { alle Zeilen j getestet }
```

Wesentliche Elemente

- Die Lösung ist endlich.
- Die Lösung wird iterativ aufgebaut. Es ist jederzeit möglich zu testen, ob die bisherige Lösung noch gültig ist (Zeile 5, 6).
- Ist eine Lösung nicht mehr möglich, wird die Teillösung auch nicht weiter verfolgt.
- Wurde eine Lösung gefunden, wird sie ausgegeben (Zeile 9, 10).
- Die Methode wird rekursiv aufgerufen, um die Lösung zu vervollständigen (Zeile 12).

Backtracking - Allgemein

Voraussetzungen für Backtracking

1. Die Lösung ist als Vektor `a[1], a[2], ...` endlicher Länge darstellbar.
2. Jedes Element `a[i]` hat eine endliche Anzahl von möglichen Werten `A[i]`.

   D. h. die Menge der möglichen Werte pro `a[i]` kann unterschiedlich sein.
3. Es gibt einen effizienten Test, ob eine Teillösung `a[1], a[2], ..., a[k]` zu einer gültigen Lösung führen kann.

Verfahren

Start: Wähle eine Teillösung `a[1]`.

Allgemein: Ist eine Teillösung basierend auf `a[1], a[2], ..., a[k-1]` noch keine Gesamtlösung, dann erweitere sie mit dem nächsten nicht ausgeschlossenen Wert `a[k]` aus `A[k]` zur neuen Teillösung `a[1], a[2], ..., a[k]`.
Falls noch nicht alle Elemente von `A[K]`, die zu keiner inkonsistenten Lösungen führen, ausgeschöpft sind, dann gehe zurück (backtrack) und wähle `a[k]` neu.
Ggf. gehe zu `a[k-1]` usw. zurück.

Es wird hier nicht gefordert, dass alle Element den gleichen Wertebereich haben. Es ist auch möglich, dass die Werte unterschiedlich sind.

✎ Übung

0.1. Auswerten logischer Ausdrücke mittels Backtracking

Bestimmen Sie für folgenden Ausdruck c - mittels Backtracking - Wahrheitswerte für die Variablen, damit der Ausdruck als Ganzes wahr wird:

$$c = (A \vee \neg B) \wedge (\neg A \vee B) \wedge (\neg A \vee \neg C) \wedge (C \vee D) \wedge (\neg C \vee \neg D)$$

Füllen Sie dazu die folgende Tabelle aus, um alle Lösungen zu finden. In der letzten Spalte geben Sie an, ob die Zeile eine Teillösung darstellt (nicht inkonsistent), keine Lösung ist bzw. sein kann, oder eine Gesamtlösung identifiziert wurde. Die Evaluation wie vieler vollständiger Belegungen wurde eingespart, wenn die Lösung gefunden wurde?

	A	B	C	D	nicht inkonsistent (T), keine Lösung (K), vollständige Lösung (L)
1	w				T
...	...	...	...	...	...
16	...	...	...	...	...

✎ Übung

0.2. Das Erfüllbarkeitsproblem

Konjunktive Normalform (KNF)

Ein logischer Ausdruck ist in KNF, wenn der Ausdruck nur als Konjunktion (UND-Verknüpfung) von Disjunktionen (ODER-Verknüpfungen) dargestellt wird. Die Negation darf nur auf Variablen angewendet werden.

Beispiel: (A ∨ B ∨ C) ∧ (¬C ∨ D)

Entwickeln Sie ein Programm — in einer Programmiersprache Ihrer Wahl — das in der Lage ist eine Formel in konjunktiver Normalform (KNF) auf Erfüllbarkeit zu prüfen.

Prüfen Sie Ihr Programm anhand der vorhergehenden Aufgabe.

※ Hinweis

Sollten Sie das Programm in Python oder Java implementieren wollen, dann können sie den entsprechenden Code im Anhang als Grundlage verwenden. Sie müssen dann nur noch die Methode `solve` implementieren. Der Code implementiert eine kleine Klassenhierarchie zur Repräsentation von logischen Ausdrücken und ermöglicht die Evaluation (`is_solution` bzw. `isSolution`) unter einer gegebenen Belegung.

Python Template

```python
from abc import abstractmethod

class Expr:

    @abstractmethod
    def is_solution(
            self,
            binding: dict["Var", bool]) -> bool | None:
        """True or False if this expression definitively
        evaluates to the respective truth value with the
        given binding or None otherwise.

        Returning a truth value does not necessarily
        require all variables to be bound to a definite
        value.

        For example, None will be returned, if the
        truth value cannot be determined with the given
        binding. E. g., if this expression represents a
        variable for which the binding has no value, None
        is returned.

        An expression such as "A ∧ B" would return True
        if A and B are both True in the
        binding and False if at least one of them is bound
        to False, and None otherwise.
        """
        raise NotImplementedError
```

```
29
30
31  class And(Expr):
32      def __init__(self, *exprs: Expr):
33          self.exprs = exprs
34
35      def is_solution(self, binding):
36          r = True
37          for expr in self.exprs:
38              e = expr.is_solution(binding)
39              if e is None:
40                  r = None
41              elif not e:
42                  return False
43          return r
44
45      def __str__(self):
46          return " ∧ ".join(str(expr) for expr in self.exprs)
47
48      def __repr__(self):
49          exprs = ", ".join(str(expr) for expr in self.exprs)
50          return "And(" + exprs + ")"
51
52
53  class Or(Expr):
54      def __init__(self, *exprs: Expr):
55          self.exprs = exprs
56
57      def is_solution(self, binding):
58          r = False
59          for expr in self.exprs:
60              e = expr.is_solution(binding)
61              if e is None:
62                  r = None
63              elif e:
64                  return True
65          return r
66
67      def __str__(self):
68          return " ∨ ".join(str(expr) for expr in self.exprs)
69
70      def __repr__(self):
71          exprs = ", ".join(repr(expr) for expr in self.exprs)
72          return "Or(" + exprs + ")"
73
74
75  class Not(Expr):
76      def __init__(self, expr: Expr):
77          self.expr = expr
78
79      def is_solution(self, binding):
80          e = self.expr.is_solution(binding)
81          if e is None:
82              return None
83          else:
84              return not e
85
86      def __str__(self):
```

```
        return f"¬{self.expr}"

    def __repr__(self):
        return "Not(" + repr(self.expr) + ")"


class Var(Expr):
    def __init__(self, name: str):
        self.name = name

    def is_solution(self, binding):
        """True or False if bound.
        None if unbound (default).
        """
        if self not in binding:
            return None
        else:
            return binding[self]

    def __str__(self):
        return self.name

    def __repr__(self):
        return 'Var("' + self.name + '")'


A = Var("a")
B = Var("b")
C = Var("c")
D = Var("d")
vars = [A, B, C, D]
""" The variables are now indexed to enable iterating over
    them in the solve function. """

expr = And(
    Or(A, B),
    Or(Not(A), B),
    Or(Not(A), Not(C)),
    Or(C, D),
    Or(Not(C), Not(D)),
)
print("Finding solutions for: " + expr.__str__())

solution: dict[Var, bool] = {}
""" Stores the current solution by mapping the name of a
    variable to its current truth value (True or False)."""


def solve(expr, vars):
```

Java Template

```

// Intended to be run using Java > 23 (Tested with Java 23 and 24)
// May require --enable-preview to do so.

interface Expr {

```

```
    /**
     * An Optional True or Optional False if this expression
     * definitively evaluates to the respective truth value
     * with the given binding or an empty Optional otherwise.
     *
     * Returning a truth value does not necessarily
     * require all variables to be bound to a definite
     * value. For example, Optional.empty will be returned,
     * if the truth value cannot be determined with the given
     * binding. E. g., if this expression represents a
     * variable for which the binding has no value,
     * Optional.empty is returned.
     *
     * An expression such as "A ∧ B" would return true
     * if A and B are both bound to "true"  and false
     * if at least one of them is bound
     * to "false", and Optional.empty otherwise.
     */
    Optional<Boolean> isSolution(Map<Var, Boolean> binding);
}

class And implements Expr {

    private final Expr[] exprs;

    And(Expr... exprs) {
        this.exprs = exprs;
    }

    public Optional<Boolean> isSolution(Map<Var, Boolean> binding) {
        Optional<Boolean> r = Optional.of(true);
        for (var expr : this.exprs) {
            final var e = expr.isSolution(binding);
            if (!e.isPresent())
                r = Optional.empty();
            else if (!e.get())
                return e;
        }
        return r;
    }

    public String toString() {
        return Arrays.stream(exprs)
                .map(Expr::toString)
                .collect(Collectors.joining(" ∧ "));
    }
}

class Or implements Expr {

    private final Expr[] exprs;

    Or(Expr... exprs) {
        this.exprs = exprs;
    }

    public Optional<Boolean> isSolution(Map<Var, Boolean> binding) {
        var r = Optional.of(false);
```

```
65          for (var expr : this.exprs) {
66              final var e = expr.isSolution(binding);
67              if (!e.isPresent())
68                  r = Optional.empty();
69              else if (e.get())
70                  return e;
71          }
72          return r;
73      }
74
75      public String toString() {
76          return Arrays.stream(exprs)
77                  .map(Expr::toString)
78                  .collect(Collectors.joining(" ∨ "));
79      }
80  }
81
82  class Not implements Expr {
83
84      private final Expr expr;
85
86      Not(Expr expr) {
87          this.expr = expr;
88      }
89
90      public Optional<Boolean> isSolution(Map<Var, Boolean> binding) {
91          final var r = expr.isSolution(binding).map(b -> !b);
92          return r;
93      }
94
95      public String toString() {
96          return "¬" + expr;
97      }
98  }
99
100 class Var implements Expr {
101
102     private final String name;
103
104     Var(String name) {
105         this.name = name;
106     }
107
108     public Optional<Boolean> isSolution(Map<Var, Boolean> binding) {
109         final var r = Optional.ofNullable(binding.get(this));
110         return r;
111     }
112
113     public String toString() {
114         return name;
115     }
116
117 }
118
119 void main() {
120     final Var A = new Var("a");
121     final Var B = new Var("b");
122     final Var C = new Var("c");
```

```
123     final Var D = new Var("d");
124     Stack<Var> vars = new Stack<>();
125     vars.addAll(Arrays.asList(new Var[] { A, B, C, D }));
126     Expr expr = new And(
127             new Or(A, B),
128             new Or(new Not(A), B),
129             new Or(new Not(A), new Not(C)),
130             new Or(C, D),
131             new Or(new Not(C), new Not(D)));
132     IO.println("Finding solutions for: " + expr.toString());
133 
134     Map<Var, Boolean> solution = new HashMap<>();
135     solve(expr, vars, solution);
136 }
137 
138 void solve(Expr expr, Stack<Var> vars, Map<Var, Boolean> solution) {
```

✎ Übung

0.3. Gruppenzuteilung

Finden Sie eine sehr gute Aufteilung von Personen (Studierenden) auf eine feste Anzahl an Gruppen, basierend auf den Präferenzen der Personen zueinander. Nutzen Sie dazu Backtracking.

Im Template ist eine initiale Aufgabenstellung hinterlegt, die es zu lösen gilt: Verteilung von 16 Studierenden auf 4 Gruppen inkl. Bewertungsmatrix (jeder Studierende hat jeden anderen mit Werten von 1 bis 10 bewertet):

https://delors.github.io/theo-algo-backtracking/code/group_assignment_template.py